

ANGULAR 21

CLIENTE REST PROFESIONAL

Consumo de API REST con CRUD Completo

GET • POST • PUT • PATCH • DELETE

Guía Paso a Paso e Incremental

Proyecto: PostManager — Gestor de Publicaciones

API: JSONPlaceholder (jsonplaceholder.typicode.com)

Dato	Detalle
Framework	Angular 21 (Standalone Components + Signals)
API REST	JSONPlaceholder — Fake REST API pública y gratuita
Operaciones CRUD	GET, POST, PUT, PATCH, DELETE
Conceptos cubiertos	Model, Service, Signal, computed, effect, Routing, HttpClient
Nivel	Intermedio
Duración estimada	10–14 horas de estudio
Stack complementario	RxJS, Angular Router, Reactive Forms

Material Universitario • Edición 2026

TABLA DE CONTENIDO

Capítulo 1: Fundamentos — ¿Qué es un Cliente REST?	4
1.1 Arquitectura Cliente-Servidor	4
1.2 Métodos HTTP y Operaciones CRUD	4
1.3 Conociendo la API: JSONPlaceholder	4
Recursos disponibles.....	5
1.4 Herramientas de Angular 21 para Consumo REST	5
Capítulo 2: Creación y Configuración del Proyecto	7
2.1 Crear el Proyecto Angular	7
2.2 Variables de Entorno	8
Capítulo 3: Model — Definición del Dominio	10
3.1 ¿Por qué Interfaces y no Clases?	10
3.2 Interfaces del Proyecto PostManager.....	10
3.3 Utility Types de TypeScript Usados	11
Capítulo 4: Service — Lógica CRUD con HttpClient + Signals.....	12
4.1 Arquitectura del Servicio.....	12
4.2 El Servicio Completo: PostService	12
4.3 Desglose del Servicio: Conceptos Explicados	17
4.3.1 Signals Privados vs. Públicos.....	17
4.3.2 computed() — Datos Derivados	17
4.3.3 effect() — Efectos Secundarios.....	18
4.3.4 Patrón de Manejo de Errores	18
Capítulo 5: Routing — Navegación entre Páginas	19
5.1 Configuración de Rutas	19
5.2 Rutas Explicadas	20
5.3 App Component con Router Outlet y Navegación	20
5.4 Leer Parámetros de Ruta.....	21
Capítulo 6: Componentes Compartidos — PostCard y PostForm.....	23
6.1 PostCard — Tarjeta de Post.....	23

Conceptos de Angular 21 Usados en PostCard.....	25
6.2 PostForm — Formulario Reutilizable	25
Capítulo 7: Páginas — Los Componentes de Vista	29
7.1 PostListComponent — Listar Todos los Posts (GET)	29
7.2 PostDetailComponent — Ver Detalle (GET por ID)	31
7.3 PostCreateComponent — Crear Post (POST)	33
7.4 PostEditComponent — Editar Post (PUT).....	33
Capítulo 8: Resumen, Mapa Conceptual y Buenas Prácticas	35
8.1 Mapa del Proyecto Completo	35
8.2 Flujo de Datos del Proyecto	35
8.3 Buenas Prácticas.....	36
8.4 Glosario.....	36

Capítulo 1: Fundamentos — ¿Qué es un Cliente REST?

Un **cliente REST** (Representational State Transfer) es una aplicación frontend que consume recursos de un servidor a través del protocolo HTTP. El cliente envía peticiones HTTP (GET, POST, PUT, DELETE) a endpoints específicos y recibe respuestas, generalmente en formato JSON.

1.1 Arquitectura Cliente-Servidor

En la arquitectura REST, el frontend (nuestro proyecto Angular) actúa como **cliente** y la API REST actúa como **servidor**. La comunicación se realiza mediante peticiones HTTP estándar, donde cada URL representa un **recurso** y cada método HTTP define la **acción** a realizar.



CONCEPTO CLAVE

REST define una convención donde cada URL identifica un recurso y cada método HTTP indica qué operación se realiza. Ejemplo: GET /posts obtiene todos los posts, POST /posts crea uno nuevo, PUT /posts/1 actualiza el post con id=1, DELETE /posts/1 lo elimina.

1.2 Métodos HTTP y Operaciones CRUD

Método HTTP	Operación CRUD	URL Ejemplo	Descripción
GET	Read (Leer)	/posts	Obtener todos los recursos
GET	Read (Leer)	/posts/1	Obtener un recurso por ID
POST	Create (Crear)	/posts	Crear un nuevo recurso
PUT	Update (Actualizar)	/posts/1	Reemplazar un recurso completo
PATCH	Update parcial	/posts/1	Modificar campos específicos
DELETE	Delete (Eliminar)	/posts/1	Eliminar un recurso

Figura: Tabla de correspondencia entre métodos HTTP y operaciones CRUD

1.3 Conociendo la API: JSONPlaceholder

JSONPlaceholder es una API REST pública y gratuita diseñada para testing y prototipado. Provee datos ficticios pero realistas, soporta las operaciones GET, POST, PUT, PATCH y DELETE, y no requiere autenticación ni registro.

Recursos disponibles

Recurso	Endpoint	Campos principales
Posts	/posts	id, title, body, userId
Comments	/comments	id, name, email, body, postId
Albums	/albums	id, title, userId
Photos	/photos	id, title, url, thumbnailUrl, albumId
Todos	/todos	id, title, completed, userId
Users	/users	id, name, email, phone, website, company

Figura: Recursos disponibles en JSONPlaceholder (base URL: <https://jsonplaceholder.typicode.com>)



NOTA IMPORTANTE

JSONPlaceholder simula las operaciones de escritura (POST, PUT, DELETE): responde con los datos correctos y códigos HTTP apropiados, pero no persiste los cambios en el servidor. Para persistencia local, usaremos Signals en nuestro servicio.

1.4 Herramientas de Angular 21 para Consumo REST

Angular 21 provee un conjunto moderno y optimizado de herramientas para interactuar con APIs REST:

Herramienta	Ubicación	Propósito
HttpClient	@angular/common/http	Realizar peticiones HTTP (GET, POST, PUT, DELETE)
provideHttpClient()	@angular/common/http	Proveer HttpClient en la app (reemplaza HttpClientModule)
withFetch()	@angular/common/http	Usar fetch() nativo en lugar de XMLHttpRequest
Signals	@angular/core	Estado reactivo local (signal, computed, effect)
RxJS Operators	rxjs	Transformar respuestas (map, catchError, retry)
inject()	@angular/core	Inyección de dependencias funcional

Figura: Herramientas clave de Angular 21 para consumo de API REST

**CONSEJO PROFESIONAL**

En Angular 21, ya no se usa HttpClientModule. Se reemplaza por la función `provideHttpClient()` con `withFetch()` en la configuración de la aplicación, lo que es más eficiente y compatible con SSR.

Capítulo 2: Creación y Configuración del Proyecto

En este capítulo crearemos el proyecto Angular 21 desde cero, configuraremos `HttpClient` y estableceremos la estructura base que usaremos durante todo el tutorial.

2.1 Crear el Proyecto Angular

PASO 1 Generar el proyecto con Angular CLI

```
1 # Crear el proyecto
2 ng new post-manager --style=scss --routing --ssr=false
3
4 # Entrar al directorio
5 cd post-manager
6
7 # Iniciar servidor de desarrollo
8 ng serve -o
```

BASH

✓ RESULTADO ESPERADO

El navegador abrirá `http://localhost:4200` con la página de bienvenida de Angular 21.

PASO 2 Configurar `HttpClient` con `provideHttpClient()`

En Angular 21, `HttpClient` se configura en `app.config.ts` usando funciones de proveedor en lugar de módulos:

```
1 // src/app/app.config.ts
2 import { ApplicationConfig } from '@angular/core';
3 import { provideRouter } from '@angular/router';
4 import {
5   provideHttpClient,
6   withFetch,
7   withInterceptors
8 } from '@angular/common/http';
9 import { routes } from './app.routes';
10
11 export const appConfig: ApplicationConfig = {
12   providers: [
13     provideRouter(routes),
14     provideHttpClient(
15       withFetch(), // Usa fetch() nativo (más moderno)
16     ),
17   ],
18 };
```

TYPESCRIPT

Figura: Configuración de proveedores en `app.config.ts` con `HttpClient` basado en `fetch`



CONCEPTO CLAVE

La función ``provideHttpClient()`` reemplaza la importación de `HttpClientModule`. La opción ``withFetch()`` configura `HttpClient` para usar la API `fetch()` nativa del navegador, que es más moderna y ofrece mejor rendimiento que `XMLHttpRequest`.

PASO 3 Estructura de carpetas del proyecto

```
1 src/app/
2 |─ models/
3 |   └─ post.model.ts    # Interfaces del dominio
4 |─ services/
5 |   └─ post.service.ts  # Servicio CRUD con HttpClient + Signals
6 |─ pages/
7 |   └─ post-list/       # Página: lista de posts
8 |   └─ post-detail/     # Página: detalle de un post
9 |   └─ post-create/     # Página: crear nuevo post
10 |   └─ post-edit/      # Página: editar post existente
11 |─ components/
12 |   └─ post-card/      # Componente: tarjeta de post
13 |   └─ post-form/      # Componente: formulario reutilizable
14 |─ app.routes.ts       # Configuración de rutas
15 |─ app.config.ts       # Configuración de la app
16 └─ app.component.ts    # Componente raíz
```

BASH

Figura: Estructura de archivos del proyecto PostManager

PASO 4 Generar archivos con Angular CLI

```
1 # Generar componentes de página
2 ng g c pages/post-list --flat
3 ng g c pages/post-detail --flat
4 ng g c pages/post-create --flat
5 ng g c pages/post-edit --flat
6
7 # Generar componentes compartidos
8 ng g c components/post-card --flat
9 ng g c components/post-form --flat
10
11 # Generar servicio
12 ng g s services/post
```

BASH

✓ RESULTADO ESPERADO

Angular CLI genera cada componente como standalone por defecto en Angular 21 e incluye su archivo de test.

2.2 Variables de Entorno

Definimos la URL base de la API en un archivo de entorno para facilitar cambios futuros:

TYPESCRIPT


```
1 // src/environments/environment.ts
2 export const environment = {
3   production: false,
4   apiUrl: 'https://jsonplaceholder.typicode.com',
5 };
```

TYPESCRIPT

```
1 // src/environments/environment.prod.ts
2 export const environment = {
3   production: true,
4   apiUrl: 'https://jsonplaceholder.typicode.com',
5 };
```

CONSEJO PROFESIONAL

Separar la URL de la API en variables de entorno permite cambiar fácilmente entre un backend de desarrollo, staging y producción sin modificar el código fuente.

Capítulo 3: Model — Definición del Dominio

El **modelo** define la forma de los datos que nuestra aplicación maneja. En TypeScript, usamos **interfaces** para describir la estructura de los objetos que recibimos de la API y los que enviamos a ella. Esto proporciona tipado fuerte, autocompletado en el IDE y detección de errores en tiempo de compilación.

3.1 ¿Por qué Interfaces y no Clases?

Aspecto	Interface	Class
Existe en runtime	No (solo compilación)	Sí (genera código JS)
Peso en el bundle	0 bytes	Añade código
Métodos	No (solo forma)	Sí
Herencia múltiple	Sí (extends múltiple)	No (una sola clase padre)
Uso ideal	DTOs, respuestas API	Entidades con lógica

Figura: Comparativa entre interfaces y clases para modelado de datos



CONCEPTO CLAVE

En Angular, las interfaces son la elección preferida para modelar datos de API porque no generan código JavaScript adicional. Son puramente descriptivas y desaparecen tras la compilación, manteniendo el bundle liviano.

3.2 Interfaces del Proyecto PostManager

PASO 1 Crear el archivo de modelo

```
1 // src/app/models/post.model.ts
2
3 /**
4  * Representa un Post completo que viene de la API
5  * (recurso ya existente con id)
6  */
7 export interface Post {
8   id: number;
9   title: string;
10  body: string;
11  userId: number;
12 }
13
14 /**
15  * Datos necesarios para crear un nuevo Post
```

TYPESCRIPT

```

16 * (sin id, lo asigna el servidor)
17 */
18 export type CreatePost = Omit<Post, 'id'>;
19
20 /**
21 * Datos para actualizar parcialmente un Post
22 * (todos los campos son opcionales)
23 */
24 export type UpdatePost = Partial<CreatePost>;
25
26 /**
27 * Comentario asociado a un Post
28 */
29 export interface Comment {
30   id: number;
31   postId: number;
32   name: string;
33   email: string;
34   body: string;
35 }
36
37 /**
38 * Estado de una operación asíncrona
39 */
40 export interface AsyncState<T> {
41   data: T | null;
42   loading: boolean;
43   error: string | null;
44 }

```

Figura: Modelo completo del dominio con Post, CreatePost, UpdatePost, Comment y AsyncState

3.3 Utility Types de TypeScript Usados

Utility Type	Sintaxis	Resultado
Omit<T, K>	Omit<Post, 'id'>	Post sin el campo id
Partial<T>	Partial<CreatePost>	Todos los campos opcionales
Pick<T, K>	Pick<Post, 'title' 'body'>	Solo title y body
Required<T>	Required<UpdatePost>	Todos los campos obligatorios
Readonly<T>	Readonly<Post>	Todos los campos de solo lectura

Figura: Utility Types de TypeScript más usados para modelado de datos



CONSEJO PROFESIONAL

El tipo `AsyncState<T>` es un patrón genérico que encapsula los tres estados posibles de una petición HTTP: cargando, datos recibidos, o error. Lo usaremos en los Signals del servicio para manejar el estado de la UI.

Capítulo 4: Service — Lógica CRUD con HttpClient + Signals

El **servicio** es la pieza central de la arquitectura. Encapsula toda la comunicación con la API REST y expone el estado de la aplicación mediante **Signals**. Los componentes nunca hablan directamente con la API; siempre lo hacen a través del servicio.

4.1 Arquitectura del Servicio

Nuestro `PostService` seguirá este patrón:

Capa	Responsabilidad	Ejemplo
Estado (Signals)	Almacenar datos reactivos	<code>signal<Post[]>([])</code>
HTTP (HttpClient)	Comunicación con la API	<code>http.get<Post[]>(url)</code>
Computados	Datos derivados	<code>computed(() => posts().length)</code>
Efectos	Reacciones a cambios	<code>effect(() => console.log(...))</code>
Métodos públicos	API del servicio	<code>getAll(), create(), update()</code>

Figura: Capas del PostService: Estado, HTTP, Computados, Efectos y Métodos

4.2 El Servicio Completo: PostService

PASO 1 Crear el servicio con todas las operaciones CRUD

```

1 // src/app/services/post.service.ts
2 import {
3   Injectable, inject, signal, computed, effect
4 } from '@angular/core';
5 import { HttpClient } from '@angular/common/http';
6 import {
7   Observable, catchError, of, tap, map, EMPTY
8 } from 'rxjs';
9 import {
10   Post, CreatePost, UpdatePost, Comment
11 } from '../models/post.model';
12 import { environment } from
13   '../environments/environment';
14
15 @Injectable({ providedIn: 'root' })
16 export class PostService {
17
18   // --- Inyección ---
19   private http = inject(HttpClient);
20   private apiUrl = `${environment.apiUrl}/posts`;

```

TYPESCRIPT

```
21
22 // =====
23 // ESTADO REACTIVO CON SIGNALS
24 // =====
25
26 /** Lista maestra de posts */
27 private _posts = signal<Post[]>([]);
28
29 /** Post actualmente seleccionado */
30 private _selectedPost = signal<Post | null>(null);
31
32 /** Indicador de carga global */
33 private _loading = signal<boolean>(false);
34
35 /** Mensaje de error */
36 private _error = signal<string | null>(null);
37
38 /** Término de búsqueda */
39 private _searchTerm = signal<string>("");
40
41 // --- Signals Públicos (solo lectura) ---
42 readonly posts = this._posts.asReadonly();
43 readonly selected = this._selectedPost.asReadonly();
44 readonly loading = this._loading.asReadonly();
45 readonly error = this._error.asReadonly();
46 readonly searchTerm = this._searchTerm.asReadonly();
47
48 // =====
49 // SIGNALS COMPUTADOS (DATOS DERIVADOS)
50 // =====
51
52 /** Cantidad total de posts */
53 readonly postCount = computed(
54   () => this._posts().length
55 );
56
57 /** Posts filtrados por búsqueda */
58 readonly filteredPosts = computed(() => {
59   const term = this._searchTerm().toLowerCase();
60   const all = this._posts();
61   if (!term) return all;
62   return all.filter(post =>
63     post.title.toLowerCase().includes(term) ||
64     post.body.toLowerCase().includes(term)
65   );
66 });
67
68 /** Cantidad de posts filtrados */
69 readonly filteredCount = computed(
70   () => this.filteredPosts().length
71 );
72
73 /** Posts agrupados por userId */
74 readonly postsByUser = computed(() => {
75   const groups = new Map<number, Post[]>();
76   this._posts().forEach(post => {
77     const list = groups.get(post.userId) ?? [];
78     list.push(post);
79     groups.set(post.userId, list);
80   });
81   return Array.from(groups.values());
82 });
```

```
80 });
81 return groups;
82 });
83
84 // =====
85 // EFFECT (REACCION A CAMBIOS)
86 // =====
87
88 private logEffect = effect(() => {
89   const count = this.postCount();
90   const err = this._error();
91   if (err) {
92     console.error('[PostService] Error:', err);
93   } else {
94     console.log(
95       `[PostService] ${count} posts cargados`
96     );
97   }
98 });
99
100 // =====
101 // MÉTODOS CRUD
102 // =====
103
104 // --- READ: Obtener todos los posts ---
105 getAll(): void {
106   this._loading.set(true);
107   this._error.set(null);
108
109   this.http.get<Post[]>(this.apiUrl).pipe(
110     tap(posts => {
111       this._posts.set(posts);
112       this._loading.set(false);
113     }),
114     catchError(err => {
115       this._error.set(
116         err.message ?? 'Error al cargar posts'
117       );
118       this._loading.set(false);
119       return EMPTY;
120     })
121   ).subscribe();
122 }
123
124 // --- READ: Obtener un post por ID ---
125 getById(id: number): void {
126   this._loading.set(true);
127   this._error.set(null);
128
129   this.http.get<Post>(`
130     ${this.apiUrl}/${id}`
131   ).pipe(
132     tap(post => {
133       this._selectedPost.set(post);
134       this._loading.set(false);
135     }),
136     catchError(err => {
137       this._error.set(
138         `Error al cargar post ${id}`
139       );
140     })
141   ).subscribe();
142 }
```

```
139   );
140   this._loading.set(false);
141   return EMPTY;
142 })
143 ).subscribe();
144 }
145
146 // --- CREATE: Crear un nuevo post ---
147 create(data: CreatePost): Observable<Post> {
148   this._loading.set(true);
149   this._error.set(null);
150
151   return this.http.post<Post>({
152     apiUrl: data
153   }).pipe(
154     tap(newPost => {
155       // Agregar al estado local con ID generado
156       const localPost: Post = {
157         ...newPost,
158         id: this._posts().length + 1,
159       };
160       this._posts.update(
161         list => [localPost, ...list]
162       );
163       this._loading.set(false);
164     }),
165     catchError(err => {
166       this._error.set('Error al crear el post');
167       this._loading.set(false);
168       return EMPTY;
169     })
170   );
171 }
172
173 // --- UPDATE: Actualizar un post (PUT) ---
174 update(id: number, data: CreatePost
175 ): Observable<Post> {
176   this._loading.set(true);
177   this._error.set(null);
178
179   return this.http.put<Post>({
180     apiUrl: `${this.apiUrl}/${id}`, data
181   }).pipe(
182     tap(updated => {
183       this._posts.update(list =>
184         list.map(p =>
185           p.id === id ? { ...p, ...data } : p
186         )
187       );
188       this._selectedPost.set(
189         { ...updated, id }
190       );
191       this._loading.set(false);
192     }),
193     catchError(err => {
194       this._error.set(
195         `Error al actualizar post ${id}`
196       );
197       this._loading.set(false);
```

```
198     return EMPTY;
199   })
200 };
201 }
202
203 // --- UPDATE PARCIAL: (PATCH) ---
204 patch(id: number, data: UpdatePost
205 ): Observable<Post> {
206   this._loading.set(true);
207
208   return this.http.patch<Post>({
209     `${this.apiUrl}/${id}`, data
210   }).pipe(
211     tap(updated => {
212       this._posts.update(list =>
213         list.map(p =>
214           p.id === id ? { ...p, ...data } : p
215         )
216     );
217     this._loading.set(false);
218   }),
219   catchError(err => {
220     this._error.set('Error en PATCH');
221     this._loading.set(false);
222     return EMPTY;
223   })
224 );
225 }
226
227 // --- DELETE: Eliminar un post ---
228 delete(id: number): Observable<void> {
229   this._loading.set(true);
230   this._error.set(null);
231
232   return this.http.delete<void>({
233     `${this.apiUrl}/${id}`
234   }).pipe(
235     tap(() => {
236       this._posts.update(list =>
237         list.filter(p => p.id !== id)
238       );
239       this._loading.set(false);
240     }),
241     catchError(err => {
242       this._error.set(
243         `Error al eliminar post ${id}`
244       );
245       this._loading.set(false);
246       return EMPTY;
247     })
248 );
249 }
250
251 // --- SEARCH: Establecer término de búsqueda ---
252 setSearch(term: string): void {
253   this._searchTerm.set(term);
254 }
255
256 // --- CLEAR: Limpiar post seleccionado ---
```



```

257 clearSelected(): void {
258   this._selectedPost.set(null);
259 }
260
261 // --- COMMENTS: Obtener comentarios ---
262 getComments(postId: number
263 ): Observable<Comment[]> {
264   return this.http.get<Comment[]>({
265     url: `${this.apiUrl}/${postId}/comments`
266   });
267 }
268 }

```

Figura: PostService completo con HttpClient, Signals, computed, effect y CRUD

4.3 Desglose del Servicio: Conceptos Explicados

4.3.1 Signals Privados vs. Públicos

El patrón `_posts` (privado) y `posts = _posts.asReadonly()` (público) garantiza **encapsulación**. Los componentes pueden leer el estado pero solo modificarlo a través de los métodos del servicio, respetando el principio de responsabilidad única.

```

1 // ENCAPSULACIÓN con Signals
2
3 // Privado: solo el servicio puede escribir
4 private _posts = signal<Post[]>([]);
5
6 // Público: los componentes solo pueden leer
7 readonly posts = this._posts.asReadonly();
8
9 // El componente usa:
10 // this.postService.posts() → lee el valor
11 // this.postService._posts → ERROR (privado)

```

TYPESCRIPT

4.3.2 computed() — Datos Derivados

Los signals computados recalculan su valor automáticamente cuando cambian sus dependencias. No necesitan suscripciones ni cleanup:

```

1 // Se recalcula SOLO cuando _posts o _searchTerm cambian
2 readonly filteredPosts = computed(() => {
3   const term = this._searchTerm().toLowerCase();
4   const all = this._posts();
5   if (!term) return all;
6   return all.filter(post =>
7     post.title.toLowerCase().includes(term)
8   );
9 });
10
11 // En la vista se usa directamente:
12 // @for (post of postService.filteredPosts(); ...) {

```

TYPESCRIPT

4.3.3 effect() — Efectos Secundarios

Un `effect()` se ejecuta cada vez que cualquiera de los signals que lee internamente cambia de valor. Es ideal para logging, analytics, o persistencia en localStorage:

```
1 // Se ejecuta cuando postCount() o _error() cambian
2 private logEffect = effect(() => {
3   const count = this.postCount();
4   const err = this._error();
5   if (err) {
6     console.error('[PostService] Error:', err);
7   } else {
8     console.log('[PostService] ${count} posts');
9   }
10 });
```

TYPESCRIPT

4.3.4 Patrón de Manejo de Errores

Cada método CRUD sigue un patrón consistente: activar loading, limpiar error, ejecutar petición, actualizar estado, desactivar loading. En caso de error, capturamos con `catchError` y actualizamos el signal de error:

```
1 // Patrón consistente para cada operación
2 operacion(): void {
3   this._loading.set(true); // 1. Activar loading
4   this._error.set(null); // 2. Limpiar error
5
6   this.http.get<T>(url).pipe(
7     tap(data => {
8       this._data.set(data); // 3. Actualizar estado
9       this._loading.set(false); // 4. Desactivar loading
10    }),
11    catchError(err => {
12      this._error.set(msg); // 3b. Guardar error
13      this._loading.set(false); // 4. Desactivar loading
14      return EMPTY; // 5. No propagar
15    })
16  ).subscribe();
17 }
```

TYPESCRIPT

Figura: Patrón estándar de manejo de estado loading/error en cada operación CRUD

⚠ ADVERTENCIA

Siempre usa ``catchError`` en las peticiones HTTP. Sin manejo de errores, una falla de red dejará la UI en estado de carga infinita y una mala experiencia de usuario.

Capítulo 5: Routing — Navegación entre Páginas

El **Router** de Angular permite navegar entre diferentes vistas de la aplicación sin recargar la página. Cada ruta mapea una URL a un componente específico.

5.1 Configuración de Rutas

PASO 1 Definir las rutas en app.routes.ts

```
1 // src/app/app.routes.ts
2 import { Routes } from '@angular/router';
3
4 export const routes: Routes = [
5   {
6     path: '',
7     redirectTo: 'posts',
8     pathMatch: 'full',
9   },
10  {
11    path: 'posts',
12    loadComponent: () =>
13      import('./pages/post-list/post-list.component')
14        .then(m => m.PostListComponent),
15    title: 'PostManager — Todos los Posts',
16  },
17  {
18    path: 'posts/new',
19    loadComponent: () =>
20      import('./pages/post-create/post-create.component')
21        .then(m => m.PostCreateComponent),
22    title: 'Crear Nuevo Post',
23  },
24  {
25    path: 'posts/:id',
26    loadComponent: () =>
27      import('./pages/post-detail/post-detail.component')
28        .then(m => m.PostDetailComponent),
29    title: 'Detalle del Post',
30  },
31  {
32    path: 'posts/:id/edit',
33    loadComponent: () =>
34      import('./pages/post-edit/post-edit.component')
35        .then(m => m.PostEditComponent),
36    title: 'Editar Post',
37  },
38  {
39    path: '**',
40    redirectTo: 'posts',
41  },
42];
```

TYPESCRIPT

Figura: Configuración completa de rutas con lazy loading y parámetros dinámicos

**CONCEPTO CLAVE**

El patrón `loadComponent: () => import(...)` activa **Lazy Loading**: el código de cada página se carga solo cuando el usuario navega a esa ruta, reduciendo el tamaño del bundle inicial y mejorando el rendimiento.

5.2 Rutas Explicadas

Ruta	Componente	Descripción	Parámetro
/posts	PostListComponent	Lista todos los posts	Ninguno
/posts/new	PostCreateComponent	Formulario para crear	Ninguno
/posts/5	PostDetailComponent	Detalle del post 5	:id = 5
/posts/5/edit	PostEditComponent	Editar el post 5	:id = 5
/**	Redirect	Cualquier otra ruta	Redirige a /posts

Figura: Mapa de rutas del proyecto PostManager

5.3 App Component con Router Outlet y Navegación

PASO 2 Configurar el componente raíz con navegación

```

1 // src/app/app.component.ts
2 import { Component } from '@angular/core';
3 import { RouterOutlet, RouterLink } from '@angular/router';
4
5 @Component({
6   selector: 'app-root',
7   standalone: true,
8   imports: [RouterOutlet, RouterLink],
9   template: `
10     <header class="navbar">
11       <a routerLink="/posts" class="logo">
12         📄 PostManager
13       </a>
14     <nav>
15       <a routerLink="/posts">Todos</a>
16       <a routerLink="/posts/new">+ Nuevo</a>
17     </nav>
18   </header>
19
20   <main class="container">
21     <router-outlet />
22   </main>
23
24   <footer class="footer">
25     <p>PostManager &copy; 2026 — Angular 21</p>

```

TYPESCRIPT

```
26 </footer>
27 `
28 styles: [
29   .navbar {
30     display: flex;
31     justify-content: space-between;
32     align-items: center;
33     padding: 1rem 2rem;
34     background: #283593;
35     color: white;
36   }
37   .navbar a {
38     color: white;
39     text-decoration: none;
40     margin-left: 1.5rem;
41   }
42   .logo {
43     font-size: 1.4rem;
44     font-weight: bold;
45   }
46   .container {
47     max-width: 960px;
48     margin: 2rem auto;
49     padding: 0 1rem;
50   }
51   .footer {
52     text-align: center;
53     padding: 1rem;
54     color: #78909C;
55     font-size: 0.85rem;
56   }
57 `]
58 })
59 export class AppComponent {}
```

Figura: App Component con barra de navegación, router-outlet y estilos básicos

5.4 Leer Parámetros de Ruta

Para acceder al parámetro `:id` en los componentes de detalle y edición, Angular 21 ofrece la función `input()` vinculada a parámetros de ruta (con `withComponentInputBinding()`), o la inyección directa de `ActivatedRoute`:

```
1 // Opción A: input() con bindToComponentInputs
2 // (requiere withComponentInputBinding() en provideRouter)
3
4 // app.config.ts
5 import { withComponentInputBinding } from '@angular/router';
6
7 provideRouter(routes, withComponentInputBinding()),
8
9 // En el componente:
10 id = input.required<string>(); // Recibe el :id
11
12
```

TYPESCRIPT

```
13 // Opción B: ActivatedRoute (clásica)
14 import { ActivatedRoute } from '@angular/router';
15
16 private route = inject(ActivatedRoute);
17
18 ngOnInit() {
19   const id = Number(
20     this.route.snapshot.paramMap.get('id')
21   );
22   this.postService.getById(id);
23 }
```

PASO 3 Actualizar app.config.ts con binding de inputs

```
1 // src/app/app.config.ts (actualizado)
2 import { ApplicationConfig } from '@angular/core';
3 import {
4   provideRouter, withComponentInputBinding
5 } from '@angular/router';
6 import {
7   provideHttpClient, withFetch
8 } from '@angular/common/http';
9 import { routes } from './app.routes';
10
11 export const appConfig: ApplicationConfig = {
12   providers: [
13     provideRouter(
14       routes,
15       withComponentInputBinding() // ← Habilita inputs
16     ),
17     provideHttpClient(withFetch()),
18   ],
19 };
```

TYPESCRIPT

Figura: Configuración actualizada con `withComponentInputBinding` para mapear parámetros de ruta a inputs

CONSEJO PROFESIONAL

Con `withComponentInputBinding()`, Angular mapea automáticamente los parámetros de ruta a los `input()` del componente. Es la forma más limpia y moderna de recibir parámetros en Angular 21.

Capítulo 6: Componentes Compartidos — PostCard y PostForm

Los componentes compartidos son piezas de UI reutilizables que se usan en múltiples páginas. Siguiendo el principio DRY (Don't Repeat Yourself), crearemos un componente de tarjeta ([PostCard](#)) y un formulario reutilizable ([PostForm](#)).

6.1 PostCard — Tarjeta de Post

El componente [PostCard](#) muestra un resumen visual de un post con acciones de ver, editar y eliminar:

```
1 // src/app/components/post-card/post-card.component.ts
2 import {
3   Component, input, output
4 } from '@angular/core';
5 import { RouterLink } from '@angular/router';
6 import { Post } from '../../models/post.model';
7
8 @Component({
9   selector: 'app-post-card',
10  standalone: true,
11  imports: [RouterLink],
12  template: `
13    <article class="card">
14      <div class="card-header">
15        <span class="user-badge">
16          User {{ post().userId }}
17        </span>
18        <span class="post-id">#{{ post().id }}</span>
19      </div>
20
21      <h3 class="card-title">
22        <a [routerLink]="['/posts', post().id]">
23          {{ post().title }}
24        </a>
25      </h3>
26
27      <p class="card-body">
28        {{ post().body | slice:0:120 }}...
29      </p>
30
31      <div class="card-actions">
32        <a [routerLink]="['/posts', post().id]"
33          class="btn btn-view">
34          📖 Ver
35        </a>
36        <a [routerLink]="['/posts', post().id, 'edit']"
37          class="btn btn-edit">
38          ✎ Editar
39        </a>
```

TYPESCRIPT

```

40   <button class="btn btn-delete"
41     (click)="onDelete.emit(post().id)">
42      Eliminar
43   </button>
44 </div>
45 </article>
46 `
47 styles: [
48   .card {
49     border: 1px solid #E0E0E0;
50     border-radius: 12px;
51     padding: 1.25rem;
52     margin-bottom: 1rem;
53     transition: box-shadow 0.2s;
54   }
55   .card:hover {
56     box-shadow: 0 4px 12px rgba(0,0,0,0.1);
57   }
58   .card-header {
59     display: flex;
60     justify-content: space-between;
61     margin-bottom: 0.5rem;
62   }
63   .user-badge {
64     background: #E8EAF6;
65     color: #283593;
66     padding: 2px 10px;
67     border-radius: 12px;
68     font-size: 0.8rem;
69     font-weight: 600;
70   }
71   .post-id { color: #9E9E9E; font-size: 0.8rem; }
72   .card-title a {
73     color: #1565C0;
74     text-decoration: none;
75   }
76   .card-body {
77     color: #546E7A;
78     line-height: 1.5;
79   }
80   .card-actions {
81     display: flex;
82     gap: 0.5rem;
83     margin-top: 1rem;
84   }
85   .btn {
86     padding: 6px 14px;
87     border: none;
88     border-radius: 8px;
89     cursor: pointer;
90     font-size: 0.85rem;
91     text-decoration: none;
92   }
93   .btn-view { background: #E3F2FD; color: #1565C0; }
94   .btn-edit { background: #FFF3E0; color: #E65100; }
95   .btn-delete { background: #FFEBEE; color: #C62828; }
96 `]
97 }}
98 export class PostCardComponent {

```



```

99 post = input.required<Post>();
100 onDelete = output<number>();
101 }

```

Figura: PostCard: componente de tarjeta con input() para datos y output() para eventos

Conceptos de Angular 21 Usados en PostCard

API	Uso	Descripción
input.required<T>()	post = input.required<Post>()	Input obligatorio tipado (reemplaza @Input)
output<T>()	onDelete = output<number>()	Output tipado (reemplaza @Output + EventEmitter)
routerLink	[routerLink]="['/posts', id]"	Navegación declarativa sin recargar página
slice pipe	body slice:0:120	Trunca texto para vista previa

Figura: APIs de Angular 21 usadas en el componente PostCard

6.2 PostForm — Formulario Reutilizable

El formulario se reutiliza tanto para **crear** como para **editar** posts. Recibe un post opcional como input y emite los datos del formulario al componente padre:

```

1 // src/app/components/post-form/post-form.component.ts
2 import {
3   Component, input, output, inject, OnInit
4 } from '@angular/core';
5 import {
6   ReactiveFormsModule, FormBuilder, Validators
7 } from '@angular/forms';
8 import { Post, CreatePost } from '../models/post.model';
9
10 @Component({
11   selector: 'app-post-form',
12   standalone: true,
13   imports: [ReactiveFormsModule],
14   template: `
15     <form [formGroup]="form"
16       (ngSubmit)="onSubmit()"
17       class="post-form">
18
19     <div class="field">
20       <label for="title">🔴 Título</label>
21       <input id="title"
22         formControlName="title"
23         placeholder="Título del post" />
24       @if (form.get('title')?.touched
25         && form.get('title')?.invalid) {

```

TYPESCRIPT

```

26   <span class="error">
27     Título requerido (mín. 5 caracteres)
28   </span>
29 }
30 </div>
31
32 <div class="field">
33   <label for="body">📝 Contenido</label>
34   <textarea id="body"
35     formControlName="body"
36     rows="6"
37     placeholder="Escribe el contenido...">
38   </textarea>
39   @if (form.get('body')?.touched
40     && form.get('body')?.invalid) {
41     <span class="error">
42       Contenido requerido (mín. 10 caracteres)
43     </span>
44   }
45 </div>
46
47 <div class="field">
48   <label for="userId">👤 ID del Autor</label>
49   <input id="userId"
50     type="number"
51     formControlName="userId"
52     placeholder="1-10" />
53 </div>
54
55 <div class="actions">
56   <button type="submit"
57     [disabled]="form.invalid"
58     class="btn-submit">
59     {{ initialData() ? '✅ Actualizar' : '➕ Crear' }}
60   </button>
61   <button type="button"
62     class="btn-cancel"
63     (click)="onCancel.emit()">
64     Cancelar
65   </button>
66 </div>
67 </form>
68 `,
69 styles: [
70   .post-form { max-width: 640px; }
71   .field {
72     margin-bottom: 1.25rem;
73   }
74   .field label {
75     display: block;
76     font-weight: 600;
77     margin-bottom: 0.4rem;
78     color: #283593;
79   }
80   .field input, .field textarea {
81     width: 100%;
82     padding: 0.6rem 0.8rem;
83     border: 2px solid #C5CAE9;

```

```
84   border-radius: 8px;
85   font-size: 1rem;
86   transition: border-color 0.2s;
87 }
88 .field input:focus, .field textarea:focus {
89   border-color: #283593;
90   outline: none;
91 }
92 .error {
93   color: #C62828;
94   font-size: 0.85rem;
95 }
96 .actions {
97   display: flex;
98   gap: 0.75rem;
99   margin-top: 1.5rem;
100 }
101 .btn-submit {
102   background: #283593;
103   color: white;
104   border: none;
105   padding: 0.7rem 1.5rem;
106   border-radius: 8px;
107   cursor: pointer;
108   font-size: 1rem;
109 }
110 .btn-submit:disabled {
111   background: #9FA8DA;
112   cursor: not-allowed;
113 }
114 .btn-cancel {
115   background: #ECEFF1;
116   border: none;
117   padding: 0.7rem 1.5rem;
118   border-radius: 8px;
119   cursor: pointer;
120 }
121 `
122 })
123 export class PostFormComponent implements OnInit {
124   initialData = input<Post | null>(null);
125   formSubmit = output<CreatePost>();
126   onCancel = output<void>();
127
128   private fb = inject(FormBuilder);
129
130   form = this.fb.nonNullable.group({
131     title: ['', [
132       Validators.required,
133       Validators.minLength(5)
134     ]],
135     body: ['', [
136       Validators.required,
137       Validators.minLength(10)
138     ]],
139     userId: [1, [
140       Validators.required,
141       Validators.min(1),
142       Validators.max(10)
```

```
143   ]],
144   });
145
146   ngOnInit(): void {
147     const data = this.initialData();
148     if (data) {
149       this.form.patchValue({
150         title: data.title,
151         body: data.body,
152         userId: data.userId,
153       });
154     }
155   }
156
157   onSubmit(): void {
158     if (this.form.valid) {
159       this.formSubmit.emit(
160         this.form.getRawValue()
161       );
162     }
163   }
164 }
```

Figura: PostForm reutilizable con Reactive Forms, validaciones y doble propósito (crear/editar)



CONSEJO PROFESIONAL

El patrón de formulario reutilizable evita duplicar código. El mismo componente PostForm funciona para crear y para editar, detectando automáticamente el modo a través del input `initialData`.

Capítulo 7: Páginas — Los Componentes de Vista

Las páginas son componentes que se renderizan en el `<router-outlet>` cuando el usuario navega a una ruta específica. Cada página consume el servicio y los componentes compartidos.

7.1 PostListComponent — Listar Todos los Posts (GET)

```
1 // src/app/pages/post-list/post-list.component.ts
2 import {
3   Component, inject, OnInit
4 } from '@angular/core';
5 import { ReactiveFormsModule, FormControl } from '@angular/forms';
6 import { PostService } from '../../services/post.service';
7 import { PostCardComponent }
8   from '../../components/post-card/post-card.component';
9
10 @Component({
11   selector: 'app-post-list',
12   standalone: true,
13   imports: [PostCardComponent, ReactiveFormsModule],
14   template: `
15     <section class="list-page">
16       <div class="list-header">
17         <h1>📄 Todos los Posts</h1>
18         <p class="count">
19           {{ service.filteredCount() }} de
20           {{ service.postCount() }} posts
21         </p>
22       </div>
23
24       <!-- Barra de búsqueda -->
25       <div class="search-bar">
26         <input [formControl]="searchCtrl"
27           placeholder="🔍 Buscar posts..."
28           class="search-input" />
29       </div>
30
31       <!-- Estado: Cargando -->
32       @if (service.loading()) {
33         <div class="loading">
34           ⌚ Cargando posts...
35         </div>
36       }
37
38       <!-- Estado: Error -->
39       @if (service.error(); as err) {
40         <div class="error-box">
41           ❌ {{ err }}
42         <button (click)="service.getAll()">
43           Reintentar
44         </button>
45       </div>
46     }
47   `
48 })
```

TYPESCRIPT

```
47
48 <!-- Lista de posts -->
49 @for (post of service.filteredPosts();
50   track post.id) {
51   <app-post-card
52     [post]="post"
53     (onDelete)="deletePost($event)" />
54   } @empty {
55     <p class="empty">
56       No se encontraron posts.
57     </p>
58   }
59 </section>
60 `,
61 styles: [
62   .list-header {
63     display: flex;
64     justify-content: space-between;
65     align-items: center;
66   }
67   .count {
68     color: #78909C;
69     font-size: 0.9rem;
70   }
71   .search-bar { margin: 1rem 0; }
72   .search-input {
73     width: 100%;
74     padding: 0.7rem 1rem;
75     border: 2px solid #C5CAE9;
76     border-radius: 10px;
77     font-size: 1rem;
78   }
79   .search-input:focus {
80     border-color: #283593;
81     outline: none;
82   }
83   .loading {
84     text-align: center;
85     padding: 2rem;
86     color: #1565C0;
87     font-size: 1.1rem;
88   }
89   .error-box {
90     background: #FFEBEE;
91     padding: 1rem;
92     border-radius: 8px;
93     color: #C62828;
94     display: flex;
95     justify-content: space-between;
96     align-items: center;
97   }
98   .empty {
99     text-align: center;
100    color: #9E9E9E;
101    padding: 3rem;
102  }
103 `]
104 })
105 export class PostListComponent implements OnInit {
```

```

106 service = inject(PostService);
107 searchCtrl = new FormControl("", {
108   nullable: true
109 });
110
111 ngOnInit(): void {
112   this.service.getAll();
113   this.searchCtrl.valueChanges.subscribe(term =>
114     this.service.setSearch(term)
115   );
116 }
117
118 deletePost(id: number): void {
119   if (confirm('¿Eliminar este post?')) {
120     this.service.delete(id).subscribe();
121   }
122 }
123 }

```

Figura: *PostListComponent: página principal con búsqueda, estados de carga/error y lista de posts*

7.2 PostDetailComponent — Ver Detalle (GET por ID)

```

1 // src/app/pages/post-detail/post-detail.component.ts
2 import {
3   Component, inject, input, OnInit, signal
4 } from '@angular/core';
5 import { RouterLink } from '@angular/router';
6 import { PostService } from '../../services/post.service';
7 import { Comment } from '../../models/post.model';
8
9 @Component({
10   selector: 'app-post-detail',
11   standalone: true,
12   imports: [RouterLink],
13   template: `
14     @if (service.loading()) {
15       <p>⌛ Cargando...</p>
16     }
17
18     @if (service.selected(); as post) {
19       <article class="detail">
20         <div class="detail-header">
21           <a routerLink="/posts">← Volver</a>
22           <a [routerLink]="['/posts', post.id, 'edit']">
23             🖋 Editar
24           </a>
25         </div>
26
27         <h1>{{ post.title }}</h1>
28         <span class="meta">
29           Post #{{ post.id }} • Autor: User {{ post.userId }}
30         </span>
31         <p class="body">{{ post.body }}</p>
32
33         <h3>💬 Comentarios ({{ comments().length }})</h3>

```

TYPESCRIPT

```

34   @for (c of comments(); track c.id) {
35     <div class="comment">
36       <strong>{{ c.name }}</strong>
37       <small>{{ c.email }}</small>
38       <p>{{ c.body }}</p>
39     </div>
40   } @empty {
41     <p>Cargando comentarios...</p>
42   }
43 </article>
44 }
45 `
46 styles: [
47   .detail-header {
48     display: flex;
49     justify-content: space-between;
50     margin-bottom: 1rem;
51   }
52   .detail-header a {
53     color: #1565C0;
54     text-decoration: none;
55   }
56   .meta {
57     color: #78909C;
58     font-size: 0.9rem;
59   }
60   .body {
61     line-height: 1.8;
62     margin: 1.5rem 0;
63     color: #37474F;
64   }
65   .comment {
66     border-left: 3px solid #C5CAE9;
67     padding: 0.75rem 1rem;
68     margin-bottom: 0.75rem;
69     background: #F5F5F5;
70     border-radius: 0 8px 8px 0;
71   }
72   .comment strong {
73     color: #283593;
74     display: block;
75   }
76   .comment small {
77     color: #78909C;
78   }
79 `]
80 })
81 export class PostDetailComponent implements OnInit {
82   // Parámetro de ruta vía input binding
83   id = input.required<string>();
84
85   service = inject(PostService);
86   comments = signal<Comment[]>([]);
87
88   ngOnInit(): void {
89     const postId = Number(this.id());
90     this.service.getByid(postId);
91     this.service.getComments(postId)
92     .subscribe(c => this.comments.set(c));

```



```
93 }
94 }
```

Figura: PostDetailComponent con detalle del post, comentarios y navegación con routerLink

7.3 PostCreateComponent — Crear Post (POST)

```
1 // src/app/pages/post-create/post-create.component.ts
2 import { Component, inject } from '@angular/core';
3 import { Router } from '@angular/router';
4 import { PostService } from '../services/post.service';
5 import { PostFormComponent }
6   from '../components/post-form/post-form.component';
7 import { CreatePost } from '../models/post.model';
8
9 @Component({
10   selector: 'app-post-create',
11   standalone: true,
12   imports: [PostFormComponent],
13   template: `
14     <h1>✚ Crear Nuevo Post</h1>
15     <app-post-form
16       (formSubmit)="onCreate($event)"
17       (onCancel)="router.navigate(['/posts'])" />
18   `
19 })
20 export class PostCreateComponent {
21   private postService = inject(PostService);
22   router = inject(Router);
23
24   onCreate(data: CreatePost): void {
25     this.postService.create(data).subscribe(() =>
26       this.router.navigate(['/posts'])
27     );
28   }
29 }
```

TYPESCRIPT

Figura: PostCreateComponent: envía datos del formulario al servicio y navega al éxito

7.4 PostEditComponent — Editar Post (PUT)

```
1 // src/app/pages/post-edit/post-edit.component.ts
2 import {
3   Component, inject, input, OnInit
4 } from '@angular/core';
5 import { Router } from '@angular/router';
6 import { PostService } from '../services/post.service';
7 import { PostFormComponent }
8   from '../components/post-form/post-form.component';
9 import { CreatePost } from '../models/post.model';
10
11 @Component({
12   selector: 'app-post-edit',
13   standalone: true,
```

TYPESCRIPT

```
14 imports: [PostFormComponent],
15 template: `
16 <h1> 🍌 Editar Post #{{ id() }}</h1>
17
18 @if (service.loading()) {
19 <p> ⌚ Cargando datos...</p>
20 }
21
22 @if (service.selected(); as post) {
23 <app-post-form
24   [initialData]="post"
25   (formSubmit)="onUpdate($event)"
26   (onCancel)="router.navigate(
27     ['/posts', id()]" />
28 }
29 `
30 }}
31 export class PostEditComponent implements OnInit {
32   id = input.required<string>();
33
34   service = inject(PostService);
35   router = inject(Router);
36
37   ngOnInit(): void {
38     this.service.getById(Number(this.id()));
39   }
40
41   onUpdate(data: CreatePost): void {
42     const postId = Number(this.id());
43     this.service.update(postId, data).subscribe(
44       () => this.router.navigate(['/posts', postId])
45     );
46   }
47 }
```

Figura: PostEditComponent: carga el post existente en el formulario y envía PUT al actualizar

✅ RESULTADO ESPERADO

Con los 4 componentes de página completos, la aplicación soporta el ciclo CRUD completo: Listar (GET), Ver detalle (GET/:id), Crear (POST), Editar (PUT) y Eliminar (DELETE) — todo conectado al servicio con Signals.

Capítulo 8: Resumen, Mapa Conceptual y Buenas Prácticas

8.1 Mapa del Proyecto Completo

Cap.	Tema	Archivos clave	Conceptos Angular 21
1	Fundamentos REST	—	HTTP, CRUD, JSON, REST
2	Crear proyecto	app.config.ts	provideHttpClient, withFetch, CLI
3	Model	post.model.ts	Interface, Omit, Partial, genéricos
4	Service	post.service.ts	HttpClient, signal, computed, effect, inject
5	Routing	app.routes.ts	provideRouter, lazy loading, withComponentInputBinding
6	Components	post-card, post-form	input(), output(), FormBuilder, Reactive Forms
7	Pages	list, detail, create, edit	OnInit, Router, subscribe, @if/@for
8	Resumen	—	Arquitectura completa

Figura: Mapa completo del tutorial: cada capítulo, archivos y conceptos cubiertos

8.2 Flujo de Datos del Proyecto

Paso	Actor	Acción	Destino
1	Usuario	Navega a /posts	Router carga PostListComponent
2	PostListComponent	Llama service.getAll()	PostService
3	PostService	http.get<Post[]>(url)	JSONPlaceholder API
4	JSONPlaceholder	Responde con JSON []	PostService
5	PostService	_posts.set(data)	Signal _posts actualizado
6	Vista	Reacciona a filteredPosts()	Renderiza PostCards
7	Usuario	Click en Eliminar	PostListComponent
8	PostListComponent	service.delete(id).subscribe()	PostService

Paso	Actor	Acción	Destino
9	PostService	http.delete + _posts.update()	Signal actualizado
10	Vista	Post desaparece de la lista	UI actualizada

Figura: Flujo completo de una operación CRUD desde el usuario hasta la API y de vuelta a la UI

8.3 Buenas Prácticas

Práctica	Razón
Centralizar HTTP en servicios	Los componentes no deben conocer URLs ni lógica HTTP
Usar signals privados + asReadonly()	Encapsulación: solo el servicio modifica el estado
computed() para datos derivados	Recalculación automática, cero suscripciones manuales
Siempre catchError en peticiones HTTP	Previene que la UI quede en estado inconsistente
Lazy loading en todas las rutas	Reduce tamaño del bundle inicial, carga más rápida
Formularios reutilizables con input/output	Principio DRY: un solo formulario para crear y editar
Usar provideHttpClient(withFetch())	Más moderno que HttpClientModule, compatible con SSR
Separar Model en interfaces	0 bytes en el bundle, tipado fuerte en compilación
input.required<T>() y output<T>()	Reemplazan @Input/@Output, más concisos y tipados
Variables de entorno para URLs	Facilita cambiar entre backends sin tocar código

Figura: Checklist de buenas prácticas para clientes REST en Angular 21

8.4 Glosario

Término	Definición
REST	Estilo arquitectónico para APIs basado en recursos y métodos HTTP
CRUD	Create, Read, Update, Delete — las 4 operaciones básicas sobre datos
HttpClient	Servicio de Angular para realizar peticiones HTTP
Signal	Contenedor de valor reactivo que notifica cambios a la UI
computed()	Signal derivado que se recalcula automáticamente

Término	Definición
effect()	Función que ejecuta efectos secundarios al cambiar signals
Observable	Flujo de datos asíncrono de RxJS (usado por HttpClient)
inject()	Función para inyectar servicios sin constructor
Lazy Loading	Carga bajo demanda de componentes al navegar
Standalone	Componente independiente sin NgModule
FormBuilder	Servicio fábrica para crear formularios reactivos
provideHttpClient()	Función que reemplaza HttpClientModule
withFetch()	Configura HttpClient para usar fetch() nativo
withComponentInputBinding()	Mapea parámetros de ruta a inputs del componente
input.required()	API moderna para declarar inputs obligatorios
output()	API moderna para declarar outputs (eventos)

Figura: Glosario completo de términos técnicos del tutorial

FIN DEL TUTORIAL

Angular 21 — Cliente REST Profesional con CRUD Completo

Material Universitario • 2026